Set

Set of numbers

In [3]:
```
myset = {1,2,3,4,5}
myset
```

Out[3]:
```
{1, 2, 3, 4, 5}
```

Length of the set

In [4]:
```
len(myset)
```

Out[4]:
```
5
```

Duplicate elements are not allowed.

In [5]:
```
my_set = {1,1,2,2,3,4,5,5}
my_set
```

Out[5]:
```
{1, 2, 3, 4, 5}
```

Set of float numbers

In [6]:
```
myset1 = {1.79,2.08,3.99,4.56,5.45}
myset1
```

Out[6]:
```
{1.79, 2.08, 3.99, 4.56, 5.45}
```

Set of Strings

In [8]:
```
myset2 = {'Asif' , 'John' , 'Tyrion'}
myset2
```

Out[8]:
```
{'Asif', 'John', 'Tyrion'}
```

Mixed datatypes

In [10]:
```
myset3 = {10,20, "Hello", (11, 22, 32)}
myset3
```

Out[10]:
```
{(11, 22, 32), 10, 20, 'Hello'}
```

set doesn't allow mutable items

In [12]:
```
myset3 = {10,20, "Hello", [11, 22, 32]}
myset3
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[12], line 1
----> 1 myset3 = {10,20, "Hello", [11, 22, 32]}
      2 myset3

TypeError: unhashable type: 'list'
```

Create an empty set

In [13]:
```
myset4 = set()
print(type(myset4))
```

```
<class 'set'>
```

In [14]:
```
my_set1 = set(('one' , 'two' , 'three' , 'four'))
my_set1
```

Out[14]:
```
{'four', 'one', 'three', 'two'}
```

Loop through a Set

In [15]:
```
myset = {'one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight'}
for i in myset:
    print(i)
```

```
one
three
two
five
eight
six
four
seven
```

In [16]:
```
for i in enumerate(myset):
    print(i)
```

```
(0, 'one')
(1, 'three')
(2, 'two')
(3, 'five')
(4, 'eight')
(5, 'six')
(6, 'four')
(7, 'seven')
```

Set Membership

In [17]:
```
myset
```

Out[17]:
```
{'eight', 'five', 'four', 'one', 'seven', 'six', 'three', 'two'}
```

Check if 'one' exist in the set

In [18]:
```
'one' in myset
```

Out[18]:
```
True
```

Check if 'ten' exist in the set

In [19]:
```
'ten' in myset
```

Out[19]:
```
False
```

In [20]:
```
if 'three' in myset: # Check if 'three' exist in the set
    print('Three is present in the set')
else:
    print('Three is not present in the set')
```

```
Three is present in the set
```

In [21]:
```
if 'eleven' in myset: # Check if 'eleven' exist in the list
    print('eleven is present in the set')
else:
    print('eleven is not present in the set')
```

```
eleven is not present in the set
```

Add & Remove Items

In [22]:
```
myset
```

Out[22]:
```
{'eight', 'five', 'four', 'one', 'seven', 'six', 'three', 'two'}
```

Add item to a set using add() method

In [23]:
```
myset.add('NINE')
myset
```

Out[23]:
```
{'NINE', 'eight', 'five', 'four', 'one', 'seven', 'six', 'three', 'two'}
```

Add multiple item to a set using update() method

In [24]:
```
myset.update(['TEN' , 'ELEVEN' , 'TWELVE'])
myset
```

Out[24]:
```
{'ELEVEN',
 'NINE',
 'TEN',
 'TWELVE',
 'eight',
 'five',
 'four',
 'one',
 'seven',
 'six',
 'three',
 'two'}
```

remove item in a set using remove() method

In [25]:
```
myset.remove('NINE')
myset
```

Out[25]:
```
{'ELEVEN',
 'TEN',
 'TWELVE',
 'eight',
 'five',
 'four',
 'one',
 'seven',
 'six',
 'three',
 'two'}
```

remove item from a set using discard() method

In [26]:
```
myset.discard('TEN')
myset
```

Out[26]:
```
{'ELEVEN',
 'TWELVE',
 'eight',
 'five',
 'four',
 'one',
 'seven',
 'six',
 'three',
 'two'}
```

Delete all items in a set

In [27]:
```
myset.clear()
myset
```

Out[27]:
```
set()
```

Delete the set object

In [28]:
```
del myset
myset
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[28], line 2
      1 del myset
----> 2 myset

NameError: name 'myset' is not defined
```

Copy Set

In [29]:
```
myset = {'one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight'}
myset
```

Out[29]:
```
{'eight', 'five', 'four', 'one', 'seven', 'six', 'three', 'two'}
```

Creating a new reference "myset1"

In [30]:
```
myset1 = myset
myset1
```

Out[30]:
```
{'eight', 'five', 'four', 'one', 'seven', 'six', 'three', 'two'}
```

In [31]:
```
id(myset) , id(myset1) # The address of both myset & myset1 will be the same
```

Out[31]:
```
(1793673321760, 1793673321760)
```

Create a copy of the list

In [32]:
```
my_set = myset.copy()
my_set
```

Out[32]:
```
{'eight', 'five', 'four', 'one', 'seven', 'six', 'three', 'two'}
```

In [33]:
```
id(my_set) # The address of my_set will be different from myset because my_set i
```

Out[33]:
```
1793673321984
```

In [34]:
```
myset.add('nine')
myset
```

Out[34]:
```
{'eight', 'five', 'four', 'nine', 'one', 'seven', 'six', 'three', 'two'}
```

In [35]:
```
myset1 # myset1 will be also impacted as it is pointing to the same Set
```

Out[35]:
```
{'eight', 'five', 'four', 'nine', 'one', 'seven', 'six', 'three', 'two'}
```

In [36]:
```
my_set # Copy of the set won't be impacted due to changes made on the original Set
```

Out[36]: {'eight', 'five', 'four', 'one', 'seven', 'six', 'three', 'two'}

Set Operation

union

In [38]:
```
s7={1,2,3,4,5}
s8={4,5,6,7,8}
s9={8,9,10}
```

Union of s7 and s8 (All elements from both sets. NO DUPLICATES)

In [8]:
```
s7.union(s8)
```

Out[8]: {1, 2, 3, 4, 5, 6, 7, 8}

In [9]:
```
s8 | s9
```

Out[9]: {4, 5, 6, 7, 8, 9, 10}

In [10]:
```
s8|s9|s7
```

Out[10]: {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}

Union of s7, s8 and s9.

In [39]:
```
s7.union(s8, s9)
```

Out[39]: {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}

In [40]:
```
s7.update(s8,s9)
s7
```

Out[40]: {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}

In [11]:
```
print(s7)
print(s8)
print(s9)
```

```
{1, 2, 3, 4, 5}
{4, 5, 6, 7, 8}
{8, 9, 10}
```

Intersection

In [ ]:
```
s7={1,2,3,4,5}
s8={4,5,6,7,8}
```

```
s9={8,9,10}
```

In [12]:
```
s7.intersection(s8)
```

Out[12]:
```
{4, 5}
```

In [13]:
```
s7.intersection(s9)
```

Out[13]:
```
set()
```

In [14]:
```
s8 & s9
```

Out[14]:
```
{8}
```

In [23]:
```
s7={1,2,3,4,5}
s8={4,5,6,7,8}
s9={8,9,10}
```

Difference

In [16]:
```
s7.difference(s8)
```

Out[16]:
```
{1, 2, 3}
```

In [17]:
```
s8 - s9
```

Out[17]:
```
{4, 5, 6, 7}
```

In [18]:
```
s9 - s8
```

Out[18]:
```
{9, 10}
```

Symmetric Difference

In [19]:
```
s7.symmetric_difference(s8)
```

Out[19]:
```
{1, 2, 3, 6, 7, 8}
```

In [20]:
```
s8 ^ s9
```

Out[20]:
```
{4, 5, 6, 7, 9, 10}
```

In [32]:
```
s7={1,2,3,4,5}
s8={4,5,6,7,8}
s9={8,9,10}
```

In [33]:
```
s7.symmetric_difference_update(s8)
```

In [34]:
```
s7
```

Out[34]:
```
{1, 2, 3, 6, 7, 8}
```

superset, subset, disjoint

In [35]:
```
a = {1,2,3,4,5,6,7,8,9}
b = {3,4,5,6,7,8}
c = {10,20,30,40}
```

In [36]:
```
a.issuperset(b)
```

Out[36]:
```
True
```

In [37]:
```
b.issubset(a)
```

Out[37]:
```
True
```

In [38]:
```
c.issubset(a)
```

Out[38]:
```
False
```

In [39]:
```
print(a)
print(b)
print(c)
```

```
{1, 2, 3, 4, 5, 6, 7, 8, 9}
{3, 4, 5, 6, 7, 8}
{40, 10, 20, 30}
```

In [40]:
```
c.isdisjoint(a)
```

Out[40]:
```
True
```

In [41]:
```
c.isdisjoint(b)
```

Out[41]:
```
True
```

In [42]:
```
a1 = {1,2,3,4,5,6,7,8,9}
b1 = {30,40,50,60,70,80}
c1 = {10,20,30,40}
```

In [44]:
```
a1.issuperset(b1)
```

Out[44]:
```
False
```

In [45]:
```
b1.issubset(a1)
```

Out[45]:
```
False
```

In [46]:
```
a1.isdisjoint(b1)
```

Out[46]:
```
True
```

Other Builtin functions

In [42]:
```
A={1, 2, 3, 4, 5, 6, 7, 8, 9}
A
```

Out[42]:
```
{1, 2, 3, 4, 5, 6, 7, 8, 9}
```

In [43]:
```
sum(A)
```

Out[43]:
```
45
```

In [44]:
```
max(A)
```

Out[44]:
```
9
```

In [45]:
```
min(A)
```

Out[45]:
```
1
```

In [46]:
```
len(A)
```

Out[46]:
```
9
```

In [47]:
```
list(enumerate(A))
```

Out[47]:
```
[(0, 1), (1, 2), (2, 3), (3, 4), (4, 5), (5, 6), (6, 7), (7, 8), (8, 9)]
```

In [48]:
```
D= sorted(A,reverse=True)
D
```

Out[48]:
```
[9, 8, 7, 6, 5, 4, 3, 2, 1]
```

In [49]:
```
sorted(D)
```

Out[49]:
```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

In [ ]: